neoway

Get connected Get smart

OpenCPU SDK

IO API Notes

Issue 1.0
Date 2025-01-23

Notice

This document is specifically for **N706B** and **N520** modules.

This document is intended for system engineers (SEs), development engineers, and test engineers.

THIS DOCUMENT PROVIDES SUPPORT FOR PRODUCT DESIGN BY THE USER. THE USER MUST DESIGN AND DEBUG THE PRODUCT ACCORDING TO THE SPECIFICATIONS AND PARAMETERS IN THIS DOCUMENT. NEOWAY ASSUMES NO RESPONSIBILITY FOR PERSONAL INJURY AND PROPERTY LOSS CAUSED BY USER'S IMPROPER OPERATION.

DUE TO PRODUCT VERSION UPDATES OR OTHER REASONS, THE CONTENT OF THIS DOCUMENT WILL BE UPDATED AS NECESSARY WITHOUT PRIOR NOTICE.

UNLESS OTHERWISE AGREED, ALL STATEMENTS, INFORMATION AND SUGGESTIONS IN THIS DOCUMENT CONSTITUTE NO WARRANTY WHETHER EXPRESS OR IMPLIED.

Neoway Technology Co., Ltd. provides customers with comprehensive technical support. For any inquiry, please contact your account manager directly or send an email to the following email address:

Sales@neoway.com

Support@neoway.com

Website: http://www.neoway.com

Contents

About This Document

Scope

This document is applicable to N706B and N520 modules.

Audience

This document is intended for system engineers (SEs), development engineers, and test engineers.

Change History

Issue	Date	Change	Changed By
1.0	2023-05	Initial release	Lai Hongxiao

Conventions

Symbol	Indication
	This warning symbol means danger. You are in a situation that could cause fatal device damage or even bodily damage.
	Means reader be careful. In this situation, you might perform an action that could result in module or product damages.
	Means note or tips for readers to use the module

1 Overview

This document introduces IO driver-related APIs in the OpenCPU SDK, primarily covering the following aspects:

- UART

  Used for UART initialization, configuration, data transmission, and callback function registration.

- I2C

  Used for I2C initialization, configuration, and data transmission.

- SPI

  Used for SPI initialization, configuration, and data transmission.

- GPIO

  Used for GPIO parameter configuration and interrupt function registration.

- ADC

  Used for voltage sampling with ADC.

2 Data Structure

2.1 UART

2.1.1 typedef enum nwy_error_e

```
typedef enum
{
    NWY_SUCCESS                  = 0,
    NWY_FAIL                     = -1,
    NWY_GEN_E_UNKNOWN            = -1,
    NWY_GEN_E_INVALID_PARA       = -2,
    NWY_GEN_E_SET_FAILED         = -3,
    NWY_GEN_E_GET_FAILED         = -4,
    NWY_GEN_E_SIM_NOT_READY      = -5,
    NWY_GEN_E_SIM_NOT_REG        = -6,
    NWY_GEN_E_ATT_NOT_ACT        = -7,
    NWY_GEN_E_GPRS_NOT_ACT       = -8,
    NWY_GEN_E_AREADY_CONNECT     = -9,
    NWY_GEN_E_PLAT_NOT_SUPPORT   = -10,
    NWY_GEN_E_AREADY_DISCONNECT  = -11,
    NWY_GEN_E_DSNET_NOT_ACTIVE   = -12,
    NWY_GEN_E_ARG_NEVER_USED     = -13,
    NWY_GEN_E_MALLOC_FAIL        = -14,
    NWY_GEN_E_RESOURE_FULL       = -15,
    NWY_GEN_E_TIMEOUT            = -16,
    NWY_GEN_E_STATCK_SIZE_SMALL  = -17,

    NWY_AT_GEN_ERR               = -100,
    NWY_AT_MSG_QUEUE_NULL        = -101,
    NWY_AT_GET_RESP_TIMEOUT      = -102,
    NWY_AT_RESP_TIMEOUT_PARA_ERR = -103,
    NWY_AT_MALLOC_ERR            = -104,
    NWY_AT_MSG_PUT_ERR           = -105,
    NWY_AT_FORWARD_REG_FULL      = -106,
    NWY_AT_UNISOL_REG_FULL       = -107,
    NWY_AT_WAIT_RESP_FAIL        = -108,
     NWY_AT_BUSY                      = -109,

    NWY_WIFI_GEN_ERR             = -200,
    NWY_WIFI_OPEN_FAILED         = -201,
    NWY_WIFI_SCAN_FAILED         = -202,
    NWY_WIFI_SCAN_TIMEOUT        = -203,
    NWY_WIFI_GETINFO_FAILED      = -204,
    NWY_WIFI_FAILED              = -205,

    NWY_FTP_GEN_ERR              = -300,
    NWY_FTP_LOGGED               = -301,
    NWY_FTP_NOT_LOGGED           = -302,
```

```
    NWY_FTP_DATA_ALREADY_OPEND      = -303,
    NWY_FTP_DSNET_NOT_ACTIVE        = -304,

    NWY_FS_GEN_ERR                 = -400,
    NWY_FS_PATH_ERR                = -401,
    NWY_FS_ACTION_FAILED            = -402,
    NWY_FS_NOT_EXIST               = -403,
    NWY_FS_OPEN_FAIL               = -404,

    NWY_SOCKET_GEN_FAILED           = -500,
    NWY_SOCKET_ACTION_FAILED        = -501,

    NWY_FOTA_WRITE_FAILED           = -600,
    NWY_FOTA_CHECK_FAILED           = -601,
    NWY_FOTA_UPDATE_RESULT_FALSE    = -602,
    NWY_FOTA_WRITE_FLAG_FAIL        = -603,

    NWY_SNTP_UPDATETIME_DNS_ERR     = -700,
    NWY_SNTP_UPDATETIME_QUERY_FAIL  = -701,
    NWY_SNTP_UPDATETIME_SOCKET_ERR  = -702,
    NWY_SNTP_BUSY   = -703,

    NWY_LBS_BUSY   = -712,

}nwy_error_e;
```

2.1.2 typedef enum nwy_flowctrl_e

```
typedef enum {
    FC_NONE = 0,    // None Flow Control
    FC_RTSCTS,     // Hardware Flow Control (rtscts)
    FC_XONXOFF    // Software Flow Control (xon/xoff)
}nwy_flowctrl_e;
```

2.1.3 typedef enum nwy_paritybits_e

```
typedef enum {
    PB_NONE = 0,
    PB_ODD,
    PB_EVEN,
    PB_SPACE,
    PB_MARK
}nwy_paritybits_e;
```

2.1.4 typedef struct nwy_uartdcb_t

```
typedef struct {
    unsigned int baudrate;
    unsigned int databits;
    unsigned int stopbits;
    nwy_paritybits_e parity;
    nwy_flowctrl_e flowctrl;
}nwy_uartdcb_t;
```

2.1.5 typedef enum nwy_baudrate_e

```
typedef enum {
```

```
    BR_300    = 300,
    BR_600    = 600,
    BR_1200   = 1200,
    BR_2400   = 2400,
    BR_4800    = 4800,
    BR_9600   = 9600,
    BR_19200  = 19200,
    BR_38400  = 38400,
    BR_57600  = 57600,
    BR_115200   = 115200,
    BR_230400   = 230400,
    BR_460800   = 460800,
    BR_921600   = 921600
}nwy_baudrate_e;
```

2.2 I2C

2.2.1 typedef enum nwy_i2c_mode_e

```
typedef enum
{
    NWY_I2C_BPS_100K, ///< normal 100Kbps
    NWY_I2C_BPS_400K, ///< fast 400Kbps
    NWY_I2C_BPS_3P5M, ///< high speed 3.5Mbps
} nwy_i2c_mode_e;
```

2.3 SPI

2.4 GPIO

2.4.1 typedef enum nwy_pin_edge_e

```
typedef enum {
PIN_NO_EDGE = 0,
PIN_RISING_EDGE,
PIN_FALLING_EDGE,
PIN_BOTH_EDGE,
PIN_HIGH_LEVEL,
PIN_LOW_LEVEL,
} nwy_pin_edge_e;
```

2.4.2 typedef enum nwy_dir_mode_e

```
typedef enum {
PIN_DIRECTION_IN = 0,
PIN_DIRECTION_OUT,
} nwy_dir_mode_e;
```

2.4.3 typedef enum nwy_value_e

```
typedef enum {
PIN_LEVEL_LOW = 0,
PIN_LEVEL_HIGH,
} nwy_value_e;
```

2.4.4 typedef enum nwy_pin_pull_e

```
typedef enum {
PIN_PULL_DISABLE,
PIN_PULL_PU,
PIN_PULL_PD,
}nwy_pin_pull_e;
```

2.5 ADC

2.5.1 typedef enum nwy_adc_e

```
typedef enum
{
   NWY_ADC_CHANNEL1 = 1,
   NWY_ADC_CHANNEL2 = 2,
    NWY_ADC_CHANNEL3 = 3,
   NWY_ADC_CHANNEL_VBAT = 4    // recommended to use this to obtain battery voltage
} nwy_adc_e;
```

2.5.2 typedef enum nwy_adc_aux_scale_e

```
typedef enum
{
   NWY_ADC_SCALE_1V250 = 0,
   NWY_ADC_SCALE_2V444 = 1,
   NWY_ADC_SCALE_3V233 = 2,
   NWY_ADC_SCALE_5V000 = 3        //MAX
} nwy_adc_aux_scale_e;
```


3 APIs

3.1 UART

The interface function definitions are located in nwy_uart_api.h for UART-related operations.

3.1.1 nwy_uart_open

Format	int nwy_uart_open (const char* name, unsigned int baudrate, nwy_flowctrl_e flowctrl);
Description	Opens the UART device and performs the initial configuration.
Parameter	name: UART device name baudrate: UART baud rate flowctrl: flow control
Return Value	Successful: UART device file descriptor Failed: < 0

3.1.2 nwy_uart_read

Format	int nwy_uart_read (int fd,unsigned char* buf, unsigned int buf_len)
Description	Reads UART data
Parameter	fd: UART device file descriptor. buf: Pointer to the buffer for storing the read data. buf_len: Size of the buffer.
Return Value	>=0: Number of bytes actually read. <0: Read failed.
Note	Currently not supported

3.1.3 nwy_uart_write

Format	int nwy_uart_write (int fd, const unsigned char* buf, unsigned int buf_len)
Description	Writes UART data
Parameter	fd: UART device file descriptor. buf: Pointer to the data to be written. buf_len: Size of the data to be written.
Return Value	>= 0: Number of bytes successfully written. <0: Write operation failed.

3.1.4 nwy_uart_dcb_set

Format	nwy_error_e nwy_uart_dcb_set(int fd, nwy_uartdcb_t *dcb)
Description	Configures the UART DCB (Device Control Block) parameters.
Parameter	fd: UART device file descriptor. dcb: Pointer to the DCB structure.
Return Value	Success: NWY_SUCCESS Failure: other error code

3.1.5 nwy_uart_dcb_get

Format	nwy_error_e nwy_uart_dcb_get(int fd, nwy_uartdcb_t *dcb)
Description	Obtains the UART DCB parameters.
Parameter	fd: UART device file descriptor. dcb: Pointer to the DCB structure.
Return Value	Success: NWY_SUCCESS Failure: other error code

3.1.6 nwy_uart_close

Format	nwy_error_e nwy_uart_close(int fd)
Description	Closes UART device
Parameter	fd: UART device file descriptor.
Return Value	Success: NWY_SUCCESS Failure: other error code

3.1.7 nwy_uart_rx_cb_register

Format	nwy_error_e nwy_uart_rx_cb_register(int fd, nwy_uart_rx_callback cb)
Description	Configures polling mode or interrupt mode and to register a callback function for UART data reading.
Parameter	fd: UART device identifier. cb: Callback function to register. If set to NULL, polling mode is used.
Return Value	Success: NWY_SUCCESS Failure: other error code

3.1.8 nwy_uart_tx_cb_register

Format	nwy_error_e nwy_uart_tx_cb_register(int fd, nwy_uart_tx_callback cb)
Description	Configures polling mode or interrupt mode and to register a callback function for UART data writing.

Parameter	fd: UART device name cb: Callback function to register. If set to NULL, polling mode is used.
Return Value	Success: NWY_SUCCESS Failure: other error code

3.1.9 nwy_uart_rx_frame_timeout_set

Format	nwy_error_e nwy_uart_rx_frame_timeout_set(int fd, unsigned int time)
Description	Sets the timeout for the UART receive data callback function.
Parameter	fd: UART device name time: Timeout period
Return Value	Success: NWY_SUCCESS Failure: other error code

3.2 I2C

The interface function definitions are located in nwy_i2c_api.h. for i2c-related operations.

3.2.1 nwy_i2c_init

Format	int nwy_i2c_init(const char * i2cDev, nwy_i2c_mode_e mode)
Description	Initializes i2c
Parameter	i2cDev: i2c device name mode: I2C operating mode
Return Value	Success: i2c device file descriptor Failure: < 0

3.2.2 nwy_i2c_read

Format	nwy_error_e nwy_i2c_read(int fd, unsigned char slaveAddr, unsigned short ofstAddr, unsigned char* ptrBuff, unsigned short length)
Description	Reads the specified data length from the destination address of the specified I2C slave device
Parameter	fd: i2c device handle slaveAddr: slave device address ofstAddr: register address ptrBuff: data storage address Length: data length
Return Value	Success: NWY_SUCCESS Failure: Other enumeration values

3.2.3 nwy_i2c_write

Format	nwy_error_e nwy_i2c_write(int fd, unsigned char slaveAddr, unsigned short ofstAddr, unsigned char* ptrData, unsigned short length)
Description	Writes data of specified length to the destination address of the specified I2C slave device.
Parameter	fd: i2c device handle slaveAddr: slave device address ofstAddr: register address ptrBuff: data storage address length: data length
Return Value	Success: NWY_SUCCESS Failure: Other enumeration values

3.2.4 nwy_i2c_deinit

Format	nwy_error_e nwy_i2c_deinit(int fd)
Description	Releases the I2C device to free its resources.
Parameter	fd: i2c device handle
Return Value	Success: NWY_SUCCESS Failure: Other enumeration values

3.3 SPI

The interface function definitions are located in nwy_spi_api.h for spi-related operations.

3.3.1 nwy_spi_init

Format	int nwy_spi_init(char *spibus, uint8_t mode, uint32_t speed, uint8_t bits)
Description	Turns on and configure the SPI bus.
Parameter	spibus: SPI bus name, support: "SPI1", "SPI2" mode: bus mode, mode0 to mode3 speed: bus speed bits: byte length
Return Value	Success: i2c device file descriptor Failure: < 0

3.3.2 nwy_spi_read

Format	nwy_error_e nwy_spi_read(int hd, void *sendaddr, void *readaddr, uint32_t len)
Description	Reads SPI data

Parameter	hd: spi bus handle readaddr: data address len: transmitted data length
Return Value	Success: NWY_SUCCESS Failure: other error code

3.3.3 nwy_spi_write

Format	nwy_error_e nwy_spi_write(int hd, void *sendaddr, uint32_t len)
Description	Writes data to the SPI device.
Parameter	hd: spi bus handle sendaddr: data address len: transmitted data length
Return Value	Success: NWY_SUCCESS Failure: other error code

3.3.4 nwy_spi_transfer

Format	nwy_error_e nwy_spi_transfer(int hd, uint8_t cs, uint8_t *tx, uint8_t *rx, uint32_t size)
Description	Transmits SPI data.
Parameter	hd: spi bus handle cs: cs parameter selection tx: Points to the pointer used to send data. rx: points to the pointer used to receive data size: bytes
Return Value	Success: NWY_SUCCESS Failure: other error code

3.3.5 nwy_spi_deinit

Format	nwy_error_e nwy_spi_deinit(int hd)
Description	Closes SPI device.
Parameter	hd: spi bus handle
Return Value	Success: offset Failure: other error code

3.4 GPIO

The interface function definitions are located in nwy_gpio_api.h for GPIO-related operations.

3.4.1 nwy_gpio_direction_get

Format	int nwy_gpio_direction_get(uint32 gpio_id)
Description	Gets the GPIO direction.
Parameter	gpio_id: GPIO ID.
Return Value	0: input 1: output -1: failure

3.4.2 nwy_gpio_direction_set

Format	nwy_error_e nwy_gpio_direction_set(uint32 gpio_id, nwy_dir_mode_e dirct)
Description	Sets the GPIO direction.
Parameter	gpio_id: GPIO ID. dirct: pin direction setting
Return Value	Success: NWY_SUCCESS Failure: other error code

3.4.3 nwy_gpio_value_get

Format	int nwy_gpio_value_get(uint32 gpio_id)
Description	Gets the pin level status of the GPIO.
Parameter	gpio_id: GPIO ID.
Return Value	0: low level 1: high level Failure: NWY_ERROR

3.4.4 nwy_gpio_value_set

Format	nwy_error_e nwy_gpio_value_set(uint32 gpio_id, nwy_value_e value)
Description	Sets the pin level state of the GPIO.
Parameter	gpio_id: GPIO ID. Value: pin level settings
Return Value	Success: NWY_SUCCESS Failure: other error code

3.4.5 nwy_gpio_pull_get

Format	int nwy_gpio_pull_get (uint32_t gpio_id)
Description	Obtains the GPIO's pull-up/pull-down property
Parameter	gpio_id: GPIO ID.

Return Value	0: no pull-up/down 1: pull-up 2: pull-down Failure: NWY_ERROR
Note	Currently not supported

3.4.6 nwy_gpio_pull_set

Format	nwy_error_e nwy_gpio_pull_set (uint32_t gpio_id, nwy_pin_pull_e pull)
Description	Sets the GPIO pull-up/drop-down property.
Parameter	gpio_id: GPIO ID. pul: pull-up/down setting
Return Value	Success: NWY_SUCCESS Failure: other error code

3.4.7 nwy_gpio_irq_register

Format	nwy_error_e nwy_gpio_irq_register (uint32_t gpio_id, nwy_pin_edge_e pin_edge, nwy_pin_pull_e pin_pull, void *eint_cb, void *wakeup_eint_cb)
Description	Sets GPIO interrupt configuration
Parameter	gpio_id: GPIO ID. pin_edge: interrupt trigger mode pin_pull: pin pull-up/down settings eint_cb: interrupt callback function. This function will be called when an interrupt is triggered normally. wakeup_eint_cb: wake-up interrupt callback function. In the sleep state, the external interrupt wake-up system will call this function.
Return Value	Success: NWY_SUCCESS Failure: other error code
Note	The wakeup_eint_cb callback is currently not supported. Set it to NULL.

3.4.8 nwy_gpio_irq_enable

Format	nwy_error_e nwy_gpio_irq_enable(uint32_t gpio_id)
Description	Enables GPIO interrupts. The callback function corresponding to the interrupt is eint_cb.
Parameter	gpio_n: GPIO ID
Return Value	Success: NWY_SUCCESS Failure: other error code

3.4.9 nwy_gpio_irq_disable

Format	nwy_error_e nwy_gpio_irq_disable(uint32_t gpio_id)

Description	Disables GPIO interrupts. The callback function corresponding to the interrupt is eint_cb.
Parameter	gpio_n: GPIO ID
Return Value	Success: NWY_SUCCESS Failure: other error code

3.4.10 nwy_gpio_irq_wakeup_enable

Format	nwy_error_e nwy_gpio_irq_wakeup_enable (uint32_t gpio_id)
Description	Enables the configured GPIO wake-up interrupt, and the corresponding callback function of this interrupt is wakeup_ein _cb.
Parameter	gpio_n: GPIO ID
Return Value	Success: NWY_SUCCESS Failure: other error code
Description	Currently not supported

3.4.11 nwy_gpio_irq_wakeup_disable

Format	nwy_error_e nwy_gpio_irq_wakeup_disable (uint32_t gpio_id)
Description	Disables configured GPIO interrupts, the callback function corresponding to the interrupt is wakeup_ein_cb.
Parameter	gpio_n: GPIO ID
Return Value	Success: NWY_SUCCESS Failure: other error code
Description	Currently not supported!

3.5 ADC

The interface function definitions are located in nwy_adc_api.h for ADC-related operations.

3.5.1 nwy_adc_get

Format	int nwy_adc_get(nwy_adc_e channel,nwy_adc_aux_scale_e scale)
Description	Gets ADC values
Parameter	Channel: ADC channel Scale: ADC range
Return Value	>= 0: The acquired voltage value. (unit mV, the error is about ± 2%) < 0: Other

